Chuck's Standard Poker Suite v1.1

A Reference for Delphi® Programmers

Licensing and Information

Chuck's Poker Suite is Copyrighted Freeware. The limits placed on its use are minimal. Here are the things you should know:

- You may not remove the copyright notice from any of the source or documentation files.
- You may not claim credit for having written the Poker Suite, its documentation, or any of the accompanying programs.
- You may use the Poker Suite to write programs for your own use or for distribution to others. This includes commercial, freeware, and shareware applications.
- The Poker Suite is royalty free – even for commercial and shareware applications.
- You may distribute complete and unmodified individual copies of the Poker Suite (and its support and demo programs) to others. You may not charge money for these copies, other than a reasonable fee to cover the cost of the media (disks, CDs etc.).
- Distributors of shareware and freeware CDs may distribute complete and unmodified copies of the Poker Suite as part of a collection of freeware and/or shareware, provided that they contact me before doing so. (I don't want any money. I wish to be contacted so that I can make sure you are distributing the latest version of the Poker Suite.)
- You may modify the Poker Suite to suit your needs, but if you wish to distribute modified copies of the source or documentation, you must contact me for permission.
- The Poker Suite comes with no warranties or guarantees either expressed or implied. If you choose to use it, you do so at your own risk.

Overview

Chuck's Standard Poker Suite is a procedural poker library for Delphi programmers. It is not Object-Oriented. The Poker Suite supports 3-card, 4-card, and 5-card poker variants. (An 5-card poker variant is one in which 5 or more cards are dealt into each hand, but only 5 cards from the hand are used to determine the hand's evaluation. For example, 7-Card Stud is a 5-card poker variant, because the best five-card hand taken from the seven cards is used to determine the winner.) The Poker Suite supports both high and low hand evaluations. It does not support wild cards. Versions of the Poker Suite are available for Delphi 1 through Delphi 2005.

New in this release are two new helper libraries. These are designed to make the Poker Suite easier to use.

The library currently consists of seven files:

1. **`caaPSCrd.pas`** – (**caa**-**P**oker-**S**uite-**C**a**rd**) – This file contains the definition of the playing card type and supporting functions.
2. **`caaPSDck.pas`** – (**caa**-**P**oker-**S**uite-**D**e**ck**) – This file contains the definition of the deck type and supporting functions.
3. **`caaPS3CH.pas`** – (**caa**-**P**oker-**S**uite-**3**-**C**ard-**H**and) – This file contains the definition of the 3-card poker hand, the 3-card poker rating engines and the supporting types and function definitions.
4. **`caaPS4CH.pas`** – (**caa**-**P**oker-**S**uite-**4**-**C**ard-**H**and) – This file contains the definition of the 4-card poker hand, the 4-card poker rating engines and the supporting types and function definitions.
5. **`caaPS5CH.pas`** – (**caa**-**P**oker-**S**uite-**5**-**C**ard-**H**and) – This file contains the definition of the 5-card poker hand, the 5-card poker rating engines and the supporting types and function definitions.
6. **`caa7Card.pas`** - (**caa**-**7**-**Card**) – This is a library helper file for those working with 7-Card Stud and Texas Hold-em. It contains the definition of a 7-card poker hand and routines to return the 5-card poker ratings and hand names.
7. **`caaOmaha.pas`** - (**caa**-**Omaha**) – This is a library helper file for those working with Omaha. It contains the definition of an Omaha poker hand and routines to return the 5-card poker ratings and hand names.

The Poker Suite also comes with a set of support and demo programs. The support programs were used during the construction of the library as validation tools. They can be useful if you choose to make modifications to the library or to help learn about the functionality of the library. The demo programs are provided as a learning aid. The library also comes with a card resource file. These programs and files include:

1. **testCard.dpr** – This program tests the playing card type. The user may enter a rank and suit using combo boxes. The program will display the name and image of the card. This validates the hand naming routine and demonstrates the use of image components with the library.
2. **testDeck.dpr** – This program tests the card deck type. The program may be used to create an ordered deck of playing cards and display the names of these cards in a list box. Once the deck is created, the user may shuffle or order the deck and display the card names in their new order.
3. **tst3Card.dpr** – This program tests the 3-Card Poker Engine. It generates all 22,100 possible 3-card poker hands in a 52-card deck and determines a high rating or low rating for each. The expected and actual numbers of each hand type are displayed in a list box.
4. **tst4Card.dpr** – This program tests the 4-Card Poker Engine. It generates all 270,725 possible 4-card poker hands in a 52-card deck and determines a high rating or low rating for each. The expected and actual numbers of each hand type are displayed in a list box.
5. **tst5Card.dpr** – This program tests the 5-Card Poker Engine. It generates all 2,598,960 possible 5-card poker hands in a 52-card deck and determines a high rating or low rating for each. The expected and actual numbers of each hand type are displayed in a list box.
6. **timeLibs.dpr** – This program times the rating engines of all three libraries and displays the results in a list box.
7. **rat3Card.dpr** – This program tests the 3-Card Poker Engine. It displays preset and random hands of cards in a list box. The information displayed includes the high hand name, high hand rating, low hand name, low hand rating, and the names of the three cards in the hand. The ratings are displayed in hexadecimal format.
8. **rat4Card.dpr** – This program tests the 4-Card Poker Engine. It displays preset and random hands of cards in a list box. The information displayed includes the high hand name, high hand rating,

low hand name, low hand rating, and the names of the four cards in the hand. The ratings are displayed in hexadecimal format.

9. **rat5Card.dpr** – This program tests the 5-Card Poker Engine. It displays preset and random hands of cards in a list box. The information displayed includes the high hand name, high hand rating, low hand name, low hand rating, and the names of the five cards in the hand. The ratings are displayed in hexadecimal format.
10. **rat57Cds.dpr** – This program tests the 5-Card Poker Engine and the 7-Card Helper Library. It displays preset and random seven-card poker hands and displays the high hand name, high hand rating, low hand name, low hand rating, and the names of the seven cards in the hand. The ratings are displayed in hexadecimal format. It uses the 5-card library to determine the results and demonstrates how to use 7-Card helper library.
11. **ratOmaha.dpr** – This program tests the 5-Card Poker Engine and the Omaha Helper Library. It displays preset and random Omaha poker hands and displays the high hand name, high hand rating, low hand name, low hand rating, and the names of the seven cards in the hand. The ratings are displayed in hexadecimal format. It uses the 5-card library to determine the results and demonstrates how to use the Omaha helper library.
12. **HandRat3.dpr** – This program tests the 3-Card Poker Engine. It allows the user to select any three cards from a standard 52-card deck and evaluate them.
13. **HandRat4**.**dpr** – This program tests the 4-Card Poker Engine. It allows the user to select any four cards from a standard 52-card deck and evaluate them.
14. **HandRat5**.**dpr** – This program tests the 5-Card Poker Engine. It allows the user to select any five cards from a standard 52-card deck and evaluate them.
15. **vidpkr.dpr** – A demo program that uses the library. This is a very simple video poker game using 5 cards.
16. **CACards01.res** – (32-Bit and .NET) or **CACRDS01.RES** in the16-bit version. This file contains card images. The naming convention used in the resource files is explained later in this document.
17. **CACards02.res** – (32-Bit and .NET) or **CACRDS02.RES** in the16-bit version. This file contains card images. The naming convention used in the resource files is explained later in this document.

18. **CACards03.res** – (32-Bit and .NET) or **CACRDS03.RES**, in the16-bit version. This file contains card images. The naming convention used in the resource files is explained later in this document.
19. **PkrSte1.doc** – The library documentation (this file) in Word 2000 format.
20. **PkrSte1.pdf** – The library documentation (this file) in Adobe PDF format.

Design Notes

This Poker Suite is an outgrowth of my original Standard Poker Library. My goals in building the Poker Suite were:

- To create a set of units that would handle 3-card and 4-card poker variants in addition to the 5-card variants supported by the original.
- To create a "cleaner" set of units. The original library is contained in a single unit. The Poker Suite is composed of five separate units. This simplifies the code in each unit and makes the suite easier to maintain and modify.
- To eliminate some of the design mistakes I made in the original. For example, the card object and hand object in the original contain unnecessary name fields.
- To create a non-object oriented poker library. Eliminating object-oriented code makes it easier to port this library to other versions of Pascal and to programming languages that do not support object oriented programming.
- To test the theory that old-fashioned structured techniques would result in higher speed.
- To use as a basis for a new object-oriented Poker Suite.

The Playing Card Type

The playing card type is defined in the unit caaPSCrd.pas. The interface section of this unit appears below. It has been formatted to fit this document and some of the comments have been removed.

```
unit caaPSCrd;

interface

type
  { string type used by getCardName Function }
```

```
TcaaCardName = string[32];

{ the fourteen possible ranks of playing cards.
  fourteen ranks are needed because the ace can rank
  either low or high. }
TcaaCardRank =
  ( caaAceLow, caaTwo, caaThree, caaFour,
    caaFive, caaSix, caaSeven, caaEight,
    caaNine, caaTen, caaJack,  caaQueen,
    caaKing, caaAceHigh );

{ The four possible suits }
TcaaCardSuit =
  ( caaClubs, caaDiamonds, caaHearts, caaSpades );

{ The card definition is simple.  }
TcaaCard = record
  CardRank : TcaaCardRank;
  CardSuit : TcaaCardSuit;
end;

{ function to return name of card. this is useful
  for debugging. }
function caaGetCardName( Card: TcaaCard ): TcaaCardName;

{ procedures to set card ranks and suits }
procedure caaSetCardRank( var Card: TcaaCard;
                              Rank: TcaaCardRank );
procedure caaSetCardSuit( var Card: TcaaCard;
                              Suit: TcaaCardSuit );
procedure caaSetCard( var Card: TcaaCard;
                          Rank: TcaaCardRank;
                          Suit: TcaaCardSuit );
```

The Card Name Type – TcaaCardName

In the 16-bit and 32-bit libraries, the card name is defined as a 32-character short string. The Delphi short string type is a Byte-Length Encoded string. The first byte contains the string length. The remaining thirty-two bytes contain character data. This type is used because it is generally faster than the long string type in 32-bit versions of Delphi. The .NET versions use a long string. In the Poker Suite, the card name is never actually stored.

The Card Rank Type – TcaaCardRank

The card rank is defined as an enumerated type containing fourteen ranks. Fourteen ranks are needed because the ace can have two different values.

The Card Suit Type – TcaaCardSuit

The card rank type is defined as an enumerated type containing four suits.

The Playing Card Type – TcaaCard

This type is a record containing fields for the **CardRank** and **CardSuit**. No other information is absolutely necessary.

The Card Name Function – function caaGetCardName

This function expects a variable or constant of type **TcaaCard**. It returns a formatted string of type **TcaaCardName** containing the name of the card. The primary purpose of this function is to make validating the library easier but it can also be used in games.

The Rank Setting Procedure – procedure caaSetCardRank

This procedure allows the programmer to set the rank of a card. It expects a variable of type **TcaaCard** and a card rank variable or constant of type **TcaaCardRank**. In the current version of the Poker Suite it is safe to directly set the rank field, but this could change if future versions.

The Suit Setting Procedure – procedure caaSetCardSuit

This procedure allows the programmer to set the suit of a card. It expects a variable of type **TcaaCard** and a card suit variable or constant of type **TcaaCardSuit**. In the current version of the Poker Suite it is safe to directly set the suit field, but this could change if future versions.

The Card Setting Procedure – procedure caaSetCard

This procedure allows the programmer to set the rank and suit of a card. It expects a variable of type **TcaaCard**, a suit variable or constant of type **TcaaCardSuit**, and a card rank variable or constant of type **TcaaCardRank**. In the current version of the Poker Suite it is safe to directly set the rank and suit fields, but this could change if future versions.

Card Testing Program – testCard.dpr

While designing the card unit, I wrote a simple testing program to validate the card type. This permits the user to choose ranks and suits using combo box controls. As each change is made, the name card name and an image of that card are displayed on the form. This program also demonstrates how to "link" the card type to images contained in a resource file.

Format of the Card Resource Files

A 16-bit card resource file is supplied with the Delphi 1 version of the Poker Suite. Three 32-bit card resource files are included with all other versions.

32-Bit: The card names in the 32-bit resource files have the following format: **SUIT_XX**, **JOKER_XX**, and **CARDBACK_XX**.

- **Standard Cards:**
  - **SUIT** contains the suit name. This can be CLUB, DIAMOND, HEART, or SPADE.
  - An underscore "_" character follows the suit name.
  - **XX** contains the card rank number. This is equal to the ordinal value of the card rank plus 1.
    - Ace = 01 – Remember that the ace can have two different values. The resource file uses the lower value.
    - Two through Nine = 02 through 09.
    - 10 = 10.
    - Jack = 11.
    - Queen = 12.
    - King = 13.
  - **Examples:**
    - King of Hearts: HEART_13
    - Three of Spades: SPADE_03
    - Ace of Clubs: CLUB_01
- **Jokers:**
  - Two Jokers are supplied, named JOKER_01 and JOKER_02.
- **Card Backs:**
  - Two card backs are supplied, named CARDBACK_01 and CARDBACK_02.

16-Bit: The card names in the 16-bit resource files have the following format: **SUIT_XX**, **JOKE_XX**, and **BACK_XX**.

- **Standard Cards:**
  - **SUIT** contains a four-character abbreviated version of the suit name. This can be “CLUB”, “DIAM”, “HEAR, or SPAD.
  - An underscore “_” character follows the suit name.
  - **XX** contains the card rank number. This is equal to the ordinal value of the card rank plus 1.
    - Ace = 01 – Remember that the ace can have two different values. The resource file uses the lower value.
    - Two through Nine = 02 through 09.
    - 10 = 10.
    - Jack = 11.
    - Queen = 12.
    - King = 13.
  - **Examples:**
    - King of Hearts: HEAR_13
    - Three of Spades: SPAD_03
    - Ace of Clubs: CLUB_01
- **Jokers:**
  - Two Jokers are supplied, named JOKE_01 and JOKE_02.
- **Card Backs:**
  - Two card backs are supplied, named BACK_01 and BACK_02.

Displaying Cards – testCard.dpr

The card-testing program is used to help validate the playing card type. It is also a tutorial on linking card images to image components. The program displays a form, which allows the user to select any of the standard fifty-two playing cards using two combo boxes. The card name and card image are then displayed on the screen.

The implementation section of the 32-bit version of the program starts below. It demonstrates how to retrieve a card name and a card image using the playing card type.

```
implementation

{$R *.DFM}
{$R CACards03.res}

var
  PlayingCard: TcaaCard;

{ function TformTestCardMain.ImageName }
function TformTestCardMain.ImageName(Card: TcaaCard): string;
begin
  case Card.CardSuit of
    caaClubs    : Result := 'CLUB_';
    caaDiamonds : Result := 'DIAMOND_';
    caaHearts   : Result := 'HEART_';
    caaSpades   : Result := 'SPADE_';
  end;
  if Card.CardRank < caaTen then Result := Result + '0';
  Result := Result + intToStr( ord( Card.CardRank ) + 1 );
end;
```

The first item of interest is the **ImageName** function. The purpose of this function is to return a name for the card image in the card resource file being used. (If you are using Delphi 1, see the file "**TSTCMAIN.PAS**" to view the source to this function. It is slightly different.) The function is fairly simple. The image name is built in three steps.

- First, a case statement is used to set up the suit prefix (with an under bar).
- If the rank is **caaAceLow** to **caaNine**, we add a '0' to the result.
- Finally, we convert the ordinal value of the card rank (+ 1) to a string and add it to the result.

Note: The default value for aces in a newly created deck is **caaAceHigh**. This would cause a problem with the code above. In this program, the **FormShow** and combo box change events ensure that all aces have the value **caaAceLow**. The video poker demo contains a modified version of the image naming code, which solves this potential problem in a different way.

```
{ procedure TformTestCardMain.ShowImage }
procedure TformTestCardMain.ShowImage(Card: TcaaCard);
var
  CardImageName: string;
  pCardName: array[0..10] of char;
begin
  { get the name of the image resource for this card }
  CardImageName := ImageName( Card );

  { convert that name to a "C" type string so as not to confuse
      the API }
  StrPCopy(pCardName,CardImageName);

  { get the image form the resource file }
  imgCard.Picture.Bitmap.Handle :=
    LoadBitmap(hInstance,pCardName);
end;
```

The **ShowImage** procedure demonstrates one method of displaying a card image contained in an image resource file. The procedure first gets the name of the image to retrieve. It converts this to a form that **LoadBitMap** can use and calls that function to display the card. (In .NET, the conversion is not needed).

```
{ procedure TformTestCardMain.FormShow }
procedure TformTestCardMain.FormShow(Sender: TObject);
begin
  { fill the card deck }
  caaBuildCardDeck( Deck );

  { set suit and rank }
  caaSetCard( PlayingCard, caaAceHigh, caaClubs );

  { update the combo boxes }
  cbxRank.ItemIndex := 0;
  cbxSuit.ItemIndex := 0;

  { update card name display }
  lblCardName.Caption  := caaGetCardName( PlayingCard );
```

```
  { update the card image display }
  if PlayingCard.CardRank = caaAceHigh then
    PlayingCard.CardRank := caaAceLow;
  ShowImage( PlayingCard );
end;
```

The **FormShow** event is used to initialize everything. It calls the **caaBuildCardDeck** procedure, passing the global variable **Deck**. If you fail to build the deck you will have a deck containing 52 club aces. Next, it sets the initial card value (as the ace of clubs) and sets the combo boxes to match this choice. (If you are ever dealt three aces of clubs in a real poker game, you should probably fold.)

It then displays the card name by calling the **caaGetCardName** function. Users of my older libraries should note that in this library card names are not stored in the card record. The code then displays the card image – but before calling the **ShowImage** routine, it first ensures that any aces are ranked as **caaAceLow**.

```
{ procedure TformTestCardMain.cbxRankChange
  Does: Updates the display based on the new rank chosen.}
procedure TformTestCardMain.cbxRankChange(Sender: TObject);
begin
  { set rank }
  caaSetCardRank( PlayingCard, TcaaCardRank(cbxRank.ItemIndex) );

  { update card name display }
  lblCardName.Caption  := caaGetCardName( PlayingCard );

  { update the card image display }
  if PlayingCard.CardRank = caaAceHigh then
    PlayingCard.CardRank := caaAceLow;
  ShowImage( PlayingCard );
end;

{ procedure TformTestCardMain.cbxSuitChange
  Does: Updates the display based on the new suit chosen.}
procedure TformTestCardMain.cbxSuitChange(Sender: TObject);
begin
  { set suit }
  caaSetCardSuit( PlayingCard, TcaaCardSuit(cbxSuit.ItemIndex) );

  { update card name display }
  lblCardName.Caption  := caaGetCardName( PlayingCard );

  { update the card image display }
  if PlayingCard.CardRank = caaAceHigh then
```

```
    PlayingCard.CardRank := caaAceLow;
  ShowImage( PlayingCard );
end;
```

The last two procedures are the combo box On Change events. The rank combo box calls **caaSetCardRank** to change the card's Rank field. The suit combo box calls **caaSetCardSuit** to change the card's Suit field. Both procedures then get a new name for the card and cause a new image to be displayed.

The Card Deck

The Card Deck is defined in the unit **caaPSDck.pas**. The interface section of this unit appears below. It has been formatted to fit this document and some of the comments and code have been removed.

```
unit caaPSDck;

interface

uses
  caaPSCrd;

const
  caaStdDeckSize = 52;

  { positions for cards in a standard ordered deck }
  caaClubAce   = 0;
  caaClubTwo   = caaClubAce + 1;
  caaClubThree = caaClubTwo + 1;
  caaClubFour  = caaClubThree + 1;

  { balance of definitions snipped }

  caaSpadeJack  = caaSpadeTen + 1;
  caaSpadeQueen = caaSpadeJack + 1;
  caaSpadeKing  = caaSpadeQueen + 1;

type
  { range of card positions in deck types }
  TcaaDeckRange = caaClubAce..caaSpadeKing;

  { the poker deck types are simple arrays of cards }
  TcaaCardDeck =
    array[TcaaDeckRange] of TcaaCard;

{ support procedures for card decks }
procedure caaBuildCardDeck( var Deck: TcaaCardDeck );
procedure caaShuffleCardDeck( var Deck: TcaaCardDeck );
```

As you can see most of the code simply sets up constants representing card positions in an ordered deck. But it also declares two data types and two procedures. These are described below.

The Deck Range Type – TcaaRange

This type defines the valid range of indexes into a card deck array using the lowest and highest card position constants.

The Card Deck Type – TcaaCardDeck

The Card Deck is simply a zero-based array of 52 Playing Cards. We can do this because this library does not support wild card poker variants and will always use a standard 52-card deck.

The Build Card Deck Procedure – caaBuildCardDeck

This procedure expects a card deck variable of type **TcaaCardDeck**. The procedure fills the card fields with rank and suit information. When complete, the card deck will be an ordered deck. Deck[0] will contain the Ace of Clubs, Deck[1] will contain the Two of Clubs, and so on until we come to Deck[51], which will contain the King of Spades. Using the card position constants you can select any card you wish from an ordered deck.

```
{ procedure caaBuildCardDeck }
procedure caaBuildCardDeck( var Deck: TcaaCardDeck );
var
  deckPos : TcaaDeckRange; { current position in the deck }
begin
  for deckPos := caaClubAce to caaSpadeKing do
  begin
    { cardPos mod 13 results in a value of 0..12. thus,
        by using the typecast, we are assigning initial ranks of
        caaAceLow.. caaKing. }
    Deck[deckPos].CardRank := TcaaCardRank( deckPos mod 13 );

    { set any aces to the default high rank. }
    if Deck[deckPos].CardRank = caaAceLow then
      Deck[deckPos].CardRank := caaAceHigh;

    { cardPos mod 13 results in a value of 0..3. by typecasting,
        we derive caaClubs..caaSpades. }
    Deck[deckPos].CardSuit := TcaaCardSuit( deckPos div 13 );
  end;
end;
```

To set the rank, the routine assigns the value (deckPos **mod** 13) to the **CardRank** field using a typecast. This returns a rank with an ordinal value from 0 to 12, which corresponds to the range **caaAceLow** through **caaKing**. Note that the value for aces is thus set to the lower of the two possible values. Therefore, the next bit of code resets any aces to the higher value.

Note: The hand evaluation engines expect the initial rank of aces to be **caaAceHigh**.

To set the rank, the routine assigns the value (deckPos **div** 13) to the **CardSuit** field using a typecast. This returns an ordinal value between 0 and 3, which corresponds to the range **caaClubs** through **caaSpades**.

The Shuffle Deck Procedure – caaShuffleDeck

This procedure expects a filled card deck variable of type **TcaaCardDeck**. The deck should be filled with valid playing cards prior to calling this procedure. The easiest way to ensure that the deck contains valid information is to call **caaBuildCardDeck**. Once the deck is shuffled the order of the cards will be random.

```
{ procedure caaShuffleCardDeck
  Expects: A valid deck of cards, which has already had values
           assigned to the cards.
  Returns: A shuffled deck of cards. }
procedure caaShuffleCardDeck( var Deck: TcaaCardDeck );
var
  ShuffleCount: integer;
  deckPos1, deckPos2 : TcaaDeckRange;
  tempCard : TcaaCard;
begin
  { shuffle the cards four times.
    to do this, move through the deck. at each position,
      select a card at another (random) position and swap
      the two cards. }
  for ShuffleCount := 1 to 4 do
    for deckPos1 := caaClubAce to caaSpadeKing do
    begin
      deckPos2 := Random(caaStdDeckSize);
      tempCard := Deck[deckPos1];
      Deck[deckPos1] := Deck[deckPos2];
      Deck[deckPos2] := tempCard;
    end;
end;
```

The deck shuffling routine is fairly simple. It loops through the deck four times. At each card position, it chooses a card at random and swaps the current card with the randomly chosen one.

Using the Card Deck – the testDeck Program

The deck-testing program is used to validate the card deck type. It displays an ordered deck of playing cards and two buttons. One button permits the user to shuffle the deck. The other permits the user to display the deck in order.

The implementation section of the **testCard** program starts below. The program is quite simple. It declares a global variable **CardDeck** of type **TcaaCardDeck**. The button click events call either the Build Deck or Shuffle Deck procedure and the local Show Deck procedure.

```
implementation

{$R *.DFM}

uses
  caaPSDck, caaPSCrd;

var
  CardDeck: TcaaCardDeck;

{ procedure TformMain.FormShow }
procedure TformMain.FormShow(Sender: TObject);
begin
  btnOrderClick( Sender );
end;
```

We want the initial display to be that of an ordered deck. Since the **btnOrderClick** event builds an ordered deck of cards and displays it, we simply call that event.

```
procedure TformMain.btnOrderClick(Sender: TObject);
begin
  { build an ordered deck of cards }
  caaBuildCardDeck( CardDeck );
  ShowDeck;
end;
```

This routine is quite simple. It calls the **caaBuildCardDeck** procedure and then displays it by calling the **ShowDeck** procedure.

```
{ procedure TformMain.btnShuffleClick }
procedure TformMain.btnShuffleClick(Sender: TObject);
begin
  { shuffle and display the deck of cards }
  caaShuffleCardDeck( CardDeck );
  ShowDeck;
end;
```

This routine is also quite simple. It calls the **caaShuffleCardDeck** procedure and then displays it by calling the **ShowDeck** procedure.

```
{ procedure TformMain.ShowDeck }
procedure TformMain.ShowDeck;
var
  deckPos: TcaaDeckRange;
  cardStr: string;
begin
  { clear the display area }
  lbxDisplay.Clear;

  { display the deck with card position numbers }
  for deckPos := caaClubAce to caaSpadeKing do
  begin
    if deckPos <= caaClubTen then
      cardStr := ' ' + IntToStr(deckPos+1) + ') '
    else
      cardStr := IntToStr(deckPos+1) + ') ';
    lbxDisplay.Items.Append( cardStr +
      caaGetCardName( CardDeck[deckPos] ) );
  end;
end;

end.
```

The **ShowDeck** procedure is the workhorse of this program. It first clears the display. Next it loops through the deck and displays a list of deck position numbers and the name of the card, which currently occupies that position. A space is inserted before the numbers 0 through 9 to improve the appearance of the display.

The Poker Hands and Poker Engines

The definitions for the three poker hand types and the three hand evaluation engines are in the files **caaPS3CH.pas**, **caaPS4CH.pas**, and **caaPS5CH.pas**. All three units provide the same functionality. The only difference lies in the size of the hand to be evaluated. It is the number of cards used to determine a winning hand that determines which library to pick – not the total number of cards dealt to each player.

Choosing a Library.

- The unit **caaPS3CH.pas** should be used for games in which **three cards** are used to determine the winning and losing hands. “Guts” (AKA Three-Toed-Pete) is probably the most popular of the three-card poker variants.
- The unit **caaPS4CH**.pas should be used for games in which **four cards** are used to determine the winning and losing hands. Somebody, somewhere must have played a four-card poker variant – but I don’t know of one.
- The unit **caaPS5CH**.pas should be used for games in which **five cards** are used to determine the winning and losing hands. This would include: “5-Card Draw”, “5-Card Stud”, “7-Card Stud”, “Texas Hold’em”, and “Omaha”. This would also be the library to choose when implementing a standard Video Poker game.

The 3-Card Hand and Poker Engine

The three-card hand and poker engine is contained in **caaPS3CH.pas**. This is the simplest of the libraries and the easiest to understand. Let's take a detailed look at this library. The source code has been formatted for this document and some of the comments have been removed. The interface section begins below.

```
interface

uses
  caaPSCrd, caaPSDck;

type
  { Tcaa3CardHandRating - this type is the core type of the
    rating system. It is implemented as a LongInt - a 32-bit
    integer. Information about the hand is mapped as described
    below:
       not used                 rank    card3  card2  card1
       3322 2222 2222 1111      1111    1100   0000   0000
       1098 7654 3210 9876      5432    1098   7654   3210
      | HO-------------------- bits --------------------LO |

    Only 16 bits are actually used. The upper two bytes are simply
    ignored. Bits 12..15 contain the hand rank. (Bit 15 is always 0)
      No Pair        = 0 -- 0000   16,440
      One Pair       = 1 -- 0001    3,744
      Flush          = 2 -- 0011    1,096
      Straight       = 3 -- 0010      720
      Trips          = 4 -- 0100       52
      Straight Flush = 5 -- 0101       44
      Royal Flush    = 6 -- 0110        4
      ----------------------------------
                      Total Hands 22,100

    The lowest 12 bits (0..11) are divided into three nibbles, which
    Contain the ordinal values of the card ranks. The order the cards
    are in is based on the rules for evaluating the hands of poker
    and is described below:

       High Hand Evaluations:
       For Three of a Kind, the cards are automatically in the
       right order.

       For one pair, the pair will occupy card3 (08..11) and card2
       (04..07)and the odd card will be in the card1 (00..03) position.

       For all straights, flushes and no pair hands, the cards will be
       Sorted with the highest card in position card3 (bits 08..11)
       and the lowest in position card1 (bits 00...03). In a baby
       straight or baby straight flush, the ace will be ranked low.
       In all other cases, the ace will be ranked high.
       Examples: K-Q-J, A-5-4, 3-2-A etc.
```

```
      Low Hand Evaluations:
      Aces are always evaluated as low cards.

      For Three of a Kind, the cards are automatically in the right
      order.

      For one pair, the pair will occupy card3 (08..11) and card2
     (04..07) and the odd card will be in the card1 (00..03) position.

      Straights and flushes do not count in low-ball evaluation. They
      are evaluated as no pair hands

      For No Pair hands the cards will be sorted with the highest
      card in position card3 (bits 08..11) and the lowest in position
      card1 (bits 00...03).

    The value stored for the card is one higher than the ordinal value
    of the card as type TcaaCardRank. This makes the rating easier
    to read. ( A two will display as a 2 instead of a 1 for example. )
    Remember this if you must directly access the rank nibbles. See
    the getCardRankName function for an example.
  }
  Tcaa3CardHandRating = LongInt;

  { range of card positions in hand }
  Tcaa3CardHandRange = 1..3;

  { Card Array }
  Tcaa3CardArray = array[Tcaa3CardHandRange] of TcaaCard;

  { Three card poker hand:
      Cards:  Contains the array of cards.
      Rating: Contains the high or low rating, depending on which
              Evaluation routine was last called. If the hand has not
              been evaluated, the value is undefined. }
  Tcaa3CardHand = record
    Cards  : Tcaa3CardArray;
    Rating : Tcaa3CardHandRating;
  end;

  { name for hand }
  Tcaa3CardHandName = string[32];

const
  { Hand Ranks used to set/test the base rank of various hands. }
  caa3CardNoPair        = $0;
  caa3CardOnePair       = $1000;
  caa3CardFlush         = $2000;
  caa3CardStraight      = $3000;
  caa3CardThreeOfAKind  = $4000;
  caa3CardStraightFlush = $5000;
  caa3CardRoyalFlush    = $6000;

  { Mask to eliminate everything but the hand ranks above. }
  caa3CardHandRanks     = $F000;
```

```
{ set the high or low value of the hand in Hand.Rating. }
procedure caa3CardSetHighValue( var Hand: Tcaa3CardHand );
procedure caa3CardSetLowValue( var Hand: Tcaa3CardHand );

{ get the name of the hand }
function caa3CardGetHandName( const Hand:Tcaa3CardHand ):
                                  Tcaa3CardHandName;
```

In this library, a poker hand is a record containing an array of three playing card records and a rating.

The card array is fairly straightforward. Each element contains a playing card record of type **TcaaCard**. The card array is not altered by the evaluation routines. They work on a copy of this data.

The rating is a simple long integer but its internal structure is a good bit more complex. You don't really need to understand the inner workings of the rating to use the library. Those who wish to may skip this section.

The hand rating is laid out as follows:

- The low order three nibbles contain the ranks of the three cards. These are arranged in a specific order by the evaluation routines. (No suit information is stored because it is not needed.) That order is determined by the standard rules of poker.
- The next nibble (bits 12 through 15) contains the base hand rank, from "No Pair" up through a "Royal Flush".
- When interpreted as a long integer, this combination of data ensures that the hand with the highest rating is the highest hand under standard poker rules.

Note (Three Card Poker):
The three-card library rates hands using the principle that the rare hands are the most valuable. That is true in almost every form of poker. However (in three-card poker) many people prefer to rate three-of-a-kind hands as the highest – even though the straight flush is mathematically more rare. Don't panic. If your game requires this, you may alter the hand ranking constants to make the three-of-a-kind hand highest. This glitch only affects three-card poker variants. All four-card and five-card poker variants (that I am aware of) use sound mathematics to determine the winning hands.

Let us examine some typical hands and their respective ratings. We will use hexadecimal numbering as this corresponds well to the card and hand rank positions.

Sample Hands:

Seven of Clubs, Ace of Diamonds, and Seven of Hearts - This is a pair of Sevens with and Ace "kicker". The High Hand Rating would be **177E**. The first number (1) tells us that this is a pair. We know that the first two cards should contain the pair, because single pair hands are rated first on the rank of the pair. The kicker is used only if two players have hands containing the same pair. In fact, the next two numbers are **77**, telling us that we have a pair of sevens. The last (and least important) card is the kicker, which in this case is an ace. Since this is a high rating, the ace is given the higher of its two possible values, an **E**. If this hand were evaluated as a low hand, the rating would be **1771**.

Five of Hearts, Three of Clubs, and Four of Diamonds – This is a five-high straight if evaluated as a high hand. The high rating would be **3543**. The first number (3) tells us that this is a straight. When straights are compared to each other the hand with the highest card is the winner, therefore the cards are ordered from highest to lowest. This gives us the **543**. If evaluated as a low hand it would be a no pair hand – a five high (sometimes called a fifty-four). The low rating would be **0543**. The leading zero tells us that the hand contains no pair (or anything higher). The remaining three cards are ordered from highest to lowest, giving us a **543**.

Two of Hearts, Ace of Hearts, and Three of Hearts – This is a three-high straight flush if evaluated as a high hand. The high rating would be **5321**. The first number (5) tells us that this is a straight flush. When straight flushes are compared to each other the hand with the highest card is the winner, therefore the cards are ordered from highest to lowest. This gives us the **321**. Note that the ace must be evaluated using its low value. It is assigned the value **1**, rather than **E** and is placed in the lowest position in the hand. If evaluated as a low hand it would be a no pair hand – a three high (sometimes called a thirty-two). The low rating would be **0321**. The leading zero tells us that the hand contains no pair (or anything higher). The remaining three cards are ordered from highest to lowest, giving us a **321**. Again, the ace is evaluated using its low card value.

Purpose of the Rating

The purpose of the rating is to supply an easy to use mechanism to differentiate between hands. This allows you to readily determine which player has the highest (or lowest) hand. If playing high, your program merely needs to get the high hand evaluation for each player's hand. The hand with the highest evaluation is the winning hand – assuming that the player has not folded.

A 3-Card Hand Rating Program – Rat3Card.dpr

The library comes with a utility program to test three-card hand ratings. This is used to validate the three-card hand type and the three-card evaluation engine. The program allows the user to display different types of hands and determine if the rating supplied by the engine is accurate.

Let's take a look at the information that is displayed by the program.

```
High: 0B87 No Pair: Jack, Eight High.
Low:  0B87 No Pair: Jack, Eight High.
Jack of Clubs
Seven of Hearts
Eight of Clubs
```

The first line contains the high hand rating and the high hand name. The rating is displayed as a hexadecimal number. This is done because the base ranking of a hand and each of the card ranks will fit in a single nibble. Each hexadecimal number also occupies one nibble (four bits). Thus there is a one to one correspondence between the card ranks and the last three numbers of the card rating (when displayed in hex). Further the base ranking for the hand is contained in the first number.

The number **0B87** is therefore quite meaningful. It tells us that the hand contains no pair or any higher-ranking hand type. We also know that the hand contains a Jack, an Eight, and a Seven. We don't know from the number what order they were dealt in, but we don't need that information for the rating. Thus, we know that the library believes we have a "No Pair" hand, with a Jack and Eight as the highest cards.

In fact, that is exactly what the text description of the hand says also and if we examine the three cards that make up the hand, we find that they are indeed a Jack, an Eight, and a Seven. We know that the rating in this case is accurate.

If we now examine the low rating, we find that it is identical to the high rating. In this case, that is exactly what we should expect.

The **Rat3Card** program is quite useful for building confidence that the 3-card library evaluates hands correctly and for testing the library should you choose to make modifications to the code.

Part of the implementation section of this 3-card hand-rating program is displayed below. This is a fairly long program and most of the routines in it are similar (and rather boring).

```
implementation

{$R *.DFM}

var
  Deck: TcaaCardDeck;

{ procedure TformMain.showHand
  Does: Displays the High Rating and High Hand Name.
        Displays the Low Rating and Low Hand Name.
        Displays the Names of the Three Cards.}
procedure TformMain.showHand( Hand: Tcaa3CardHand );
var
  tmpStr: string;
begin
  { high hand name and rating }
  caa3CardSetHighValue( Hand );
  tmpStr := 'High: ' +
         IntToHex( Hand.Rating, 4 ) +
         ' ' +
         caa3CardGetHandName( Hand );
  lbxDisplay.Items.Add( tmpStr );

  { low hand name and rating }
  caa3CardSetLowValue( Hand );
  tmpStr := 'Low:  ' +
         IntToHex( Hand.Rating, 4 ) +
         ' ' +
         caa3CardGetHandName( Hand );
  lbxDisplay.Items.Add( tmpStr );

  { names of cards in order }
  tmpStr := caaGetCardName( Hand.Cards[1] );
```

```
  lbxDisplay.Items.Add( tmpStr );
  tmpStr := caaGetCardName( Hand.Cards[2] );
  lbxDisplay.Items.Add( tmpStr );
  tmpStr := caaGetCardName( Hand.Cards[3] );
  lbxDisplay.Items.Add( tmpStr );
  lbxDisplay.Items.Add( '' );
end;
```

The **showHand** procedure displays hands of cards and both their high and low ratings. It starts by getting the high rating for the hand and builds a display string (**tmpStr**). This display string contains the rating of the hand in hexadecimal format followed by the name of the hand. It uses the **caa3CardSetHighValue** procedure to get the high rating and the **caa3CardGetHandName** procedure to get the hand name (as a high hand).

Note: The hand name procedures in the libraries expect a hand of cards with the rating set. For example, you should always call **caa4CardSetHighValue** or **caa4CardSetLowValue** routine before calling **caa4CardGetHandName** - if you are using the 4-Card library. This is because the hand name routines use the rating (rather than the card array) to determine the hand name.

Next, the routine gets the low rating for the hand and again builds a display string (**tmpStr**). This display string contains the low rating of the hand in hexadecimal format followed by the name of the hand. It uses the **caa3CardSetLowValue** procedure to get the low rating and the **caa3CardGetHandName** procedure to get the hand name (as a low hand).

It then gets the names of the individual cards using **caaGetCardName** and displays them below the hand ratings and hand names.

```
{ procedure TformMain.btnRandomClick }
procedure TformMain.btnRandomClick(Sender: TObject);
var
  Hand: Tcaa3CardHand;
  count: integer;
  deckPos1,
  deckPos2,
  deckPos3: TcaaDeckRange;
begin
  lbxDisplay.Clear;

  for count := 1 to 10 do
```

```
  begin
    deckPos1 := Random( caaStdDeckSize );

    { second card must be different from first card }
    repeat
      deckPos2 := Random( caaStdDeckSize );
    until deckPos2 <> deckPos1;

    { third card must be different from first two }
    repeat
      deckPos3 := Random( caaStdDeckSize );
    until (deckPos3 <> deckPos1) and
          (deckPos3 <> deckPos2);

    Hand.cards[1] := Deck[deckPos1];
    Hand.cards[2] := Deck[deckPos2];
    Hand.cards[3] := Deck[deckPos3];
    showHand( Hand );
  end;
end;
```

The **btnRandomClick** event procedure creates ten random three-card hands and passes them to the **showHand** procedure for display. There are several ways to do this. Perhaps the simplest would be to set up a new deck of cards and shuffle it. Then copy the first three cards in the deck into the first hand, the second group of three cards into the second hand and continue with this pattern until all ten hands are full.

In this case, I chose to select three random card positions from the deck for each hand. This is a bit trickier. The second and third card positions are chosen inside **repeat/until** loops to prevent duplications. When all three positions are selected, the cards are copied into the hand and the hand is then passed to the **showHand** procedure.

```
{ procedure TformMain.btnStraightFlushesClick }
procedure TformMain.btnStraightFlushesClick(Sender:
TObject);
var
  Hand: Tcaa3CardHand;
  cardPos: TcaaDeckRange;
begin
  lbxDisplay.Clear;

  lbxDisplay.Items.Add( 'The next six hands are the
same.');
```

```
  lbxDisplay.Items.Add( '' );

  Hand.Cards[1] := Deck[caaClubQueen];
  Hand.Cards[2] := Deck[caaClubKing];
  Hand.Cards[3] := Deck[caaClubAce];
  showHand( Hand );

  Hand.Cards[1] := Deck[caaClubQueen];
  Hand.Cards[2] := Deck[caaClubAce];
  Hand.Cards[3] := Deck[caaClubKing];
  showHand( Hand );

  Hand.Cards[1] := Deck[caaClubKing];
  Hand.Cards[2] := Deck[caaClubQueen];
  Hand.Cards[3] := Deck[caaClubAce];
  showHand( Hand );

  Hand.Cards[1] := Deck[caaClubKing];
  Hand.Cards[2] := Deck[caaClubAce];
  Hand.Cards[3] := Deck[caaClubQueen];
  showHand( Hand );

  Hand.Cards[1] := Deck[caaClubAce];
  Hand.Cards[2] := Deck[caaClubQueen];
  Hand.Cards[3] := Deck[caaClubKing];
  showHand( Hand );

  Hand.Cards[1] := Deck[caaClubAce];
  Hand.Cards[2] := Deck[caaClubKing];
  Hand.Cards[3] := Deck[caaClubQueen];
  showHand( Hand );

  lbxDisplay.Items.Add(
    'The rest of these are different.');
  lbxDisplay.Items.Add( '' );

  for cardPos := caaClubAce to caaClubJack do
  begin
    Hand.Cards[1] := Deck[cardPos];
    Hand.Cards[2] := Deck[cardPos+2];
    Hand.Cards[3] := Deck[cardPos+1];
    showHand( Hand );
  end;
end;
```

The **btnStraightFlushesClick** event is typical of much of the rest of this program and is the last routine in this program that we will look at. The

procedure generates a total of 17 straight flushes. The first six are all the same hand. They differ only in card order. This is done to verify that the order in which cards are dealt ***does not*** affect the ratings returned by the library. To select cards for the hands, I used the card position constants from **caaPSDck.pas**. After each hand is complete, the **showHand** procedure is called to display it.

To create the remaining hands, I select three cards in sequence from the same suit. This is done in a loop, which selects a value for the **cardPos** index variable. This value runs from **caaClubAce** (the position for the Ace of Clubs in an ordered deck) through the position for a Jack of Clubs (**caaClubJack**). The second and third cards are offset two and one positions from the **cardPos** index respectively. This will ensure that we evaluate all the possible straight flushes in a single suit. Again, after each hand is created, the **showHand** procedure is called to display it.

We won't examine the other click events, as they are all fairly similar. If you want to look at them, they are contained in **Rat3main.pas**.

Another 3-Card Hand Rating Program – HandRat3.dpr

A second 3-card hand rating utility is also provided with the library. This utility permits the user to individually select three cards and display the hand and ratings. This allows the user to test the library and is also useful for discovering the actual ratings returned for specific hands.

The implementation section of the program begins below.

```
implementation

{$R *.DFM}

uses
  caaPSCrd, caaPSDck, caaPS3CH;

var
  Hand: Tcaa3CardHand;
  Deck: TcaaCardDeck;
```

The program first declares two global variables for the hand and deck that will be used.

```
{ procedure FormCreate
  Does:  Builds an ordered deck of playing cards for use by
the program. }
procedure TfrmRateHands.FormCreate(Sender: TObject);
begin
  caaBuildCardDeck( Deck );
end;
```

In the **formCreate** procedure the **caaBuildCardDeck** routine is called to build an ordered deck of cards. We will want an ordered deck so that we may easily select the cards we want.

```
{ procedure FormShow
  Does:  Sets the On Change event handler for all the combo
boxes.
         Sets the item index for all combo boxes.
         Sets the initial hand values to match the combo
boxes. }
procedure TfrmRateHands.FormShow(Sender: TObject);
begin
  cbxRankCard1.OnChange := cbxRankCardChange;
  cbxRankCard2.OnChange := cbxRankCardChange;
  cbxRankCard3.OnChange := cbxRankCardChange;

  cbxSuitCard1.OnChange := cbxSuitCardChange;
  cbxSuitCard2.OnChange := cbxSuitCardChange;
  cbxSuitCard3.OnChange := cbxSuitCardChange;

  cbxRankCard1.ItemIndex := 0;
  cbxRankCard2.ItemIndex := 0;
  cbxRankCard3.ItemIndex := 0;

  cbxSuitCard1.ItemIndex := 0;
  cbxSuitCard2.ItemIndex := 0;
  cbxSuitCard3.ItemIndex := 0;

  Hand.Cards[1].CardRank := caaAceHigh;
  Hand.Cards[2].CardRank := caaAceHigh;
  Hand.Cards[3].CardRank := caaAceHigh;

  Hand.Cards[1].CardSuit := caaClubs;
  Hand.Cards[2].CardSuit := caaClubs;
  Hand.Cards[3].CardSuit := caaClubs;
end;
```

The **formShow** event is used to initialize the combo boxes and hand variable. First, the **OnChange** events for the card rank and card suit combo boxes are set. All the rank combo boxes use the same **OnChange** event handler. The suit combo boxes also use a single **OnChange** event handler. This gets rid of the problems associated with duplicate code.

The **ItemIndex** property of each combo box is then set to zero (0). This sets the index to the first item. In the rank box, this is the Ace. In the suit box, this is Clubs. Finally, each card in the hand is also set to the Ace of Clubs – to match the initial settings in the boxes.

```
{ procedure btnEvaluateClick }
procedure TfrmRateHands.btnEvaluateClick(Sender: TObject);
var
  DeckPos: TcaaDeckRange;
  HighRating, LowRating: Tcaa3CardHandRating;
begin
  lbxHandDisplay.Clear;

  { first card }
  DeckPos := TcaaDeckRange(( cbxSuitCard1.ItemIndex * 13) +
                           ( cbxRankCard1.ItemIndex ));
  Hand.Cards[1] := Deck[DeckPos];
  lbxHandDisplay.Items.Add( '1) ' +
    caaGetCardName( Hand.Cards[1] ) );

  { second card }
  DeckPos := TcaaDeckRange(( cbxSuitCard2.ItemIndex * 13) +
                           ( cbxRankCard2.ItemIndex ) );
  Hand.Cards[2] := Deck[DeckPos];
  lbxHandDisplay.Items.Add( '2) ' +
    caaGetCardName( Hand.Cards[2] ) );

  { third card }
  DeckPos := TcaaDeckRange(( cbxSuitCard3.ItemIndex * 13) +
                           ( cbxRankCard3.ItemIndex ) );
  Hand.Cards[3] := Deck[DeckPos];
  lbxHandDisplay.Items.Add( '3) ' +
    caaGetCardName( Hand.Cards[3] ) );

  caa3CardSetHighValue( Hand );
  HighRating := Hand.Rating;
  lbxHandDisplay.Items.Add( caa3CardGetHandName( Hand ) );

  caa3CardSetLowValue( Hand );
```

```
  LowRating := Hand.Rating;
  lbxHandDisplay.Items.Add( caa3CardGetHandName( Hand ) );

  txtHighRating.Text := IntToHex( HighRating, 4 );
  txtLowRating.Text := IntToHex( LowRating, 4 );
end;
```

The **btnEvaluateClick** event is the workhorse of the program. It displays the hand and the ratings. It starts by clearing the list box that displaying the card and hand names.

Next, it gets the position in an ordered deck of the first card in the hand. It does this using the rank and suit values for the first card from the "Card 1" combo boxes. To calculate the deck position, multiply the **ItemIndex** for the suit combo box by 13 and then add the **ItemIndex** of the rank combo box to the result. (The typecast isn't actually needed in the above code. But it does serve to remind me of what I'm trying to accomplish.) The routine next gets the card at the position that has just been calculated. Using the card, the routine then displays the card number and name. The same thing is done for cards two and three.

Once the hand is filled, the routine then gets and saves the high hand rating. Calling **caa3CardSetHighValue** and storing the value in **Hand.Rating** in the variable **HighRating** accomplishes this. With the high rating set, the routine gets the name of the hand as a high hand. To do this, it calls **caa3CardGetHandName**. The high hand name is then displayed.

Next the routine gets and saves the low hand rating. The **caa3CardSetLowValue** routine is called. The new value in **Hand.Rating** is stored in the variable **LowRating**. With the low rating set, the routine gets and displays the name of the hand as a low hand.

Finally, the high and low ratings are displayed in hexadecimal format.

```
{ procedure cbxRankCardChange
  Does:  Changes the rank of the appropriate card. }
procedure TfrmRateHands.cbxRankCardChange(Sender: TObject);
begin
  with Sender as TComboBox do
    Hand.Cards[Tag+1].CardRank := TcaaCardRank(ItemIndex);
end;
```

The **cbxRankCardChange** event handler handles the **OnChange** event for all the rank combo boxes. If you examine the rank combo boxes using the object inspector, you will see that the Tag properties for each rank combo box are different. They are set to 0, 1, and 2 respectively. Using the tag property, we are able to distinguish between the combo boxes and thus change the rank of the appropriate card. This is done by setting the card rank for the card at **Hand.Cards[Tag + 1]** equal to the **ItemIndex** of the combo box (typecast as **TcaaCardRank**). There is another way of doing this. The **Tag** properties could have been set in code in the **formShow** event and there is no inherent reason to use zero based indexing for the **Tags**.

Note for .NET users: In the .NET versions of Delphi, the **Tag** property is no longer an integer. In VCL.NET it is a Variant. The code above will not compile – because a Variant is not a valid array indexer. The event handler for Delphi 8 and Delphi 2005 (.NET) is somewhat different. See below.

```
Rank Change event for .NET:

procedure TfrmRateHands.cbxRankCardChange(Sender: TObject);
var
  cardTag: integer;  { for .NET }
begin
  with Sender as TComboBox do
  begin
    cardTag := Tag;
    Hand.Cards[cardTag + 1].CardRank :=
      TcaaCardRank(ItemIndex);
  end;
end;
```

The fix for .NET is fairly simple. Declare a **cardTag** variable of type integer and assign the **Tag** to it. Then use **cardTag** as the array index.

```
{ procedure cbxSuitCardChange
  Does:  Changes the suit of the appropriate card. }
procedure TfrmRateHands.cbxSuitCardChange(Sender: TObject);
begin
  with Sender as TComboBox do
    Hand.Cards[Tag+1].CardSuit := TcaaCardSuit(ItemIndex);
end;
```

The suit change event handler is almost identical to the rank change event handler and works in the same general way.

The 3-Card Hand Testing Program – Tst3Card.dpr

This program allows the user to validate the 3-card library by demonstrating that it returns the proper number of each possible type of poker hand. The user is presented with a form containing a list box and three buttons. The library can be tested for high or low hands.

When the user presses the “Run Test High” button, the program evaluates all 22,100 possible three-card hands in a standard deck. It then displays the expected and actual numbers of each possible hand type as well as the actual and expected total number of hands. If the actual and expected numbers match, the user can be confident that the library accurately rates the various high hand types. The “Run Test Low” button performs the same test using low hands. The implementation section of the main unit appears below.

```
uses
  caaPSCrd, caaPSDck, caaPS3CH;

procedure TformMain.btnClearClick(Sender: TObject);
begin
  lbxDisplay.Clear;
end;
```

The event handler for the **btnClearClick** event simply clears the list box used to display results.

```
{ procedure btnRunTestHighClick }
procedure TformMain.btnRunTestHighClick(Sender: TObject);
var
  i, c1, c2, c3: Integer;
  limit: Integer;
  Stats: array[1..7] of LongInt;
```

```
  handCount: LongInt;
  Hand: Tcaa3CardHand;
  Deck: TcaaCardDeck;
begin
  { set up holders for specific hand statistics }
  for i := 1 to 7 do
    Stats[i] := 0;

  { build an ordered deck }
  caaBuildCardDeck( Deck );
  { caaShuffleCardDeck( Deck ); }

  { initialize hand count }
  handCount := 0;
```

The **btnRunTestHighClick** event handles the evaluation and display of the results for high hands. There are seven possible types of high hands. These are Royal Flush, Straight Flush, Three of a Kind, Straight, Flush, Pair, and Other. The procedure first declares the variables needed and then initializes the **Stats** array, which will contain the actual results. It then builds an ordered deck and sets the **HandCount** variable to 0. You can run the test using a shuffled deck by un-commenting the call to **caaShuffleCardDeck**. The results should be the same.

```
  { initialize loop limit }
  limit := caaStdDeckSize;
  for c1 := 0 to limit - 3 do
    for c2 := c1 + 1 to limit - 2 do
      for c3 := c2 + 1 to limit - 1 do
      begin
        Inc(handCount);
        Hand.Cards[1] := Deck[c1];
        Hand.Cards[2] := Deck[c2];
        Hand.Cards[3] := Deck[c3];
```

The next task is to create all possible three-card hands. This is done using a series of nested loops. The first card in each hand will range from the card in deck position 0 to the card in deck position 49. The second card will come from positions 1 through 50. The third card will come from positions 2 through 51. Once each card position is selected for a hand, the hand count variable is incremented. This keeps a running total of the number of hands. When the test is completed, 22,100 hands should have been evaluated.

Each card in the hand is then assigned a card from the deck using the deck positions contained in the loop counters.

```
        caa3CardSetHighValue( Hand );

        if Hand.Rating >= caa3CardRoyalFlush then
           Inc(Stats[1])
        else if Hand.Rating >= caa3CardStraightFlush then
           Inc(Stats[2])
        else if Hand.Rating >= caa3CardThreeOfAKind then
           Inc(Stats[3])
        else if Hand.Rating >= caa3CardStraight then
           Inc(Stats[4])
        else if Hand.Rating >= caa3CardFlush then
           Inc(Stats[5])
        else if Hand.Rating >= caa3CardOnePair then
           Inc(Stats[6])
        else
           Inc(Stats[7]);
      end; { for }
```

The hand is then evaluated (as a high hand) using the **caa3CardSetHighValue** procedure. The **Rating** field of the **Hand** will now contain the high rating. The **Stats** array is then updated using if statements to compare the **Rating** to the base hand rank constants declared in **caaPS3CH.pas**. This gives us the total number of actual hands of each type.

```
  with lbxDisplay do
  begin
    Items.Add('Three Card Library - High Hands');
    Items.Add('Hand Evaluation  Expected Actual');
    Items.Add('Royal Flushes:          4 ' +
      IntToStr(Stats[1]));
    Items.Add('Straight Flushes:      44 ' +
      IntToStr(Stats[2]));
    Items.Add('Three of a Kinds:      52 ' +
      IntToStr(Stats[3]));
    Items.Add('Straights:            720 ' +
      IntToStr(Stats[4]));
    Items.Add('Flushes:             1096 ' +
      IntToStr(Stats[5]));
    Items.Add('Pairs:               3744 ' +
      IntToStr(Stats[6]));
    Items.Add('Other:              16440 ' +
      IntToStr(Stats[7]));
```

```
    Items.Add('Total Hands:        22100 ' +
      IntToStr(handCount));
    Items.Add('');
  end;
end;
```

The routine then displays the results. The low hand test is similar to the high hand test but is somewhat simpler because there are only three types of low hands.

The 4-Card Hand and Poker Engine

The four-card hand and poker engine is contained in **caaPS4CH.pas**. A set of test programs (similar to the ones for the three-card library) comes with the poker suite. These projects will not be covered here as they work in the same manner as their three-card counterparts. The source code for the four-card versions may be viewed by loading the following projects into Delphi.

- HandRat4.dpr
- Rat4Card.dpr
- Tst4Card.dpr

The 5-Card Hand and Poker Engine

The five-card hand and poker engine is contained in **caaPS5CH.pas**. A set of test programs (similar to the ones for the three-card library) comes with the poker suite. These projects will not be covered here as they work in the same manner as their three-card counterparts. The source code for the five-card versions may be viewed by loading the following projects into Delphi.

- HandRat5.dpr
- Rat5Card.dpr
- Tst5Card.dpr

The 7-Card Helper Library

The seven-card helper library is contained in **caa7Card.pas**. This library is designed to make it easier to code seven-card poker games such as 7-Card Stud and Texas Hold-em. The interface section of the library is on the next page.

```
interface

uses
  caaPSCrd, caaPS5CH;

type
  { Card Arrays }
  Tcaa7CardArray = array[1..7] of TcaaCard;

  { 7 Card Hand record }
  Tcaa7CardHand = record
    Cards  : Tcaa7CardArray;
    HighRating : Tcaa5CardHandRating;
    LowRating: Tcaa5CardHandRating;
  end;

{ 7-Card Stud and Texas Hold-em: Helper routines -------- }
{ Hand rating procedures for 7-Card Hands }
procedure caa7CardSetHighValue(var Hand: Tcaa7CardHand);
procedure caa7CardSetLowValue(var Hand: Tcaa7CardHand);

{ Hand name functions for 7-Card Hands }
function caa7CardGetHighHandName(const Hand:Tcaa7CardHand):
    Tcaa5CardHandName;
function caa7CardGetLowHandName(const Hand:Tcaa7CardHand):
    Tcaa5CardHandName;
```

The unit defines a seven-card hand type, containing an array of seven cards and both a high and low rating. It then declares two procedures that set the high and low ratings for the hand. Finally, it declares two functions, which return the high or low hand names.

```
{ 7-Card Stud and Texas Hold-em: Helper routines ------- }
{ procedure caa7CardSetHighValue }
procedure caa7CardSetHighValue( var Hand: Tcaa7CardHand );
var
  c1, c2, c3, c4, c5: integer; { card positions }
  tmpHand: Tcaa5CardHand;
begin
  { create all 21 possible five card hands and evaluate
    each of them, while retaining to high and low ratings.
    initialize the high ratings with value that is lower
    than possible. }
  Hand.HighRating := 0;
```

```
  { loop through cards, creating all five card hands }
  for c1 := 1 to 3 do
    for c2 := c1 + 1 to 4 do
      for c3 := c2 + 1 to 5 do
        for c4 := c3 + 1 to 6 do
          for c5 := c4 + 1 to 7 do
          begin
            { set the temporary 5-card evaluation hand
              equal to the five chosen cards }
            tmpHand.Cards[1] := Hand.Cards[c1];
            tmpHand.Cards[2] := Hand.Cards[c2];
            tmpHand.Cards[3] := Hand.Cards[c3];
            tmpHand.Cards[4] := Hand.Cards[c4];
            tmpHand.Cards[5] := Hand.Cards[c5];

            { get a high rating }
            caa5CardSetHighValue( tmpHand );

            { keep highest value }
            if tmpHand.Rating > Hand.HighRating then
              Hand.HighRating := tmpHand.Rating;

          end; { for c5 }
end;
```

The high rating procedure creates all possible five-card hand combinations from the seven cards and evaluates each of them, retaining the highest rating.

The first step in doing this is to set the **HighRating** field of the hand to zero. This ensures that at least one of the ratings will be saved. We next build each of the twenty-one possible hands using a set of nested "**for**" loops to choose five-card hands from the seven-card hand. Once we have the five cards, they are assigned to a temporary five-card hand and passed to the 5-Card Poker Engine by calling **caa5CardSetHighValue**. Finally, the **Rating** field of the temporary hand is compared to the current value of the seven-card hand's **HighRating** field. If the temporary rating is higher than the current high rating, the **HighRating** field is updated.

The **caa7CardSetLowValue** procedure (see caa7Card.pas) works in essentially the same manner.

```
{ function caa7CardGetHighHandName }
function caa7CardGetHighHandName(const Hand:Tcaa7CardHand):
         Tcaa5CardHandName;
var
  tmpHand: Tcaa5CardHand;
begin
  tmpHand.Rating := Hand.HighRating;
  result := caa5CardGetHandName( tmpHand );
end;
```

The high hand naming function is pretty simple. It creates a temporary variable of Tcaa5CardHand type and copies the **HighRating** to this temporary hand's **Rating** field. It then calls **caa5CardGetHandName** using the temporary hand. You will notice that the cards are not copied to the temporary hand.

The 5 of 7-Card Hand Rating Program – Rat57Cds.dpr

The 5-Card library comes with an additional validation program that we will take a look at. This program is significantly different than it was in the original version. This version utilizes the new caa7Card helper library. This project is very similar to the **Rat3Card** project examined earlier but it differs in two respects. First, five cards are used to get the rating. Second, the total number of cards in the hand is seven.

```
procedure TformMain.showHand( Hand: Tcaa7CardHand );
var
  tmpStr: string;
begin
  { set the high and low ratings }
  caa7CardSetHighValue( Hand );
  caa7CardSetLowValue( Hand );

  { display high name and rating }
  tmpStr := 'High: ' +
    ratingToString( Hand.HighRating ) +
    '  (Hex:' +
    IntToHex( Hand.HighRating, 6 ) +
    ')  ' +
    caa7CardGetHighHandName( Hand );
  lbxDisplay.Items.Add( tmpStr );
```

Let’s take a look at the ShowHand method, which is used to display the hand data. The routine first gets the high and low hand ratings by calling the rating routines from the seven-card helper library.

It next converts the high hand rating into a string containing fairly easy to read information about the hand and card ranks. It also converts the rating into a hexadecimal number containing the same information. It then uses the get high name function from the seven-card helper library to get the name of the hand. Finally it displays this.

```
  { display low name and rating }
  tmpStr := 'Low:  ' +
    ratingToString( Hand.LowRating ) +
    '  (Hex:' +
    IntToHex( Hand.LowRating, 6 ) +
    ')  ' +
    caa7CardGetLowHandName( Hand );
  lbxDisplay.Items.Add( tmpStr );
```

After displaying the high hand information, the low hand information is also displayed.

```
  { display the names of the cards in order }
  tmpStr := caaGetCardName( Hand.Cards[1] );
  lbxDisplay.Items.Add( ' 1) ' + tmpStr );
  tmpStr := caaGetCardName( Hand.Cards[2] );
  lbxDisplay.Items.Add( ' 2) ' + tmpStr );
  tmpStr := caaGetCardName( Hand.Cards[3] );
  lbxDisplay.Items.Add( ' 3) ' + tmpStr );
  tmpStr := caaGetCardName( Hand.Cards[4] );
  lbxDisplay.Items.Add( ' 4) ' + tmpStr );
  tmpStr := caaGetCardName( Hand.Cards[5] );
  lbxDisplay.Items.Add( ' 5) ' + tmpStr );
  tmpStr := caaGetCardName( Hand.Cards[6] );
  lbxDisplay.Items.Add( ' 6) ' + tmpStr );
  tmpStr := caaGetCardName( Hand.Cards[7] );
  lbxDisplay.Items.Add( ' 7) ' + tmpStr );
  lbxDisplay.Items.Add( '' );
end;
```

The routine then displays the names of each of the seven cards in the hand.

The Video Poker Demo – vidpkr.dpr

The Poker Suite comes with a minimalist video poker program designed to demonstrate the use of the library. We'll take a detailed look at this program and how it works. I would suggest that you compile and run this program before continuing.

When the program is first run, the user will see five card backs in the upper portion of the window. Beneath this are the message labels (black with lime letters) and the control buttons. The "Bet" button is active. The "Draw" button is inactive.

After the player has clicked the "Bet" button a hand of cards will be dealt. The "Bet" button becomes inactive. The "Draw" button is now active. The player can select which cards to hold by clicking on the card graphics. Once the player has selected the cards to hold and clicked the "Draw" button, the program replaces the cards that were not held and evaluates the current hand. A message in yellow tells the player the result of the last wager and the type of hand that resulted.

The "Bet" button is reactivated and the "Draw" button is deactivated. The code for the video poker demo begins below.

```
unit vpkrmain;

(*
 * unit vpkrMain Copyright 2005, Charles A. Appel Jr.
 *)

interface

uses
  Windows, Messages, SysUtils, Classes, Graphics, Controls,
  Forms, Dialogs, StdCtrls, ExtCtrls;

type
  TformMain = class(TForm)
    btnBet: TButton;
    btnDraw: TButton;
    lblBet: TLabel;
    lblCredits: TLabel;
    lblInfo: TLabel;
    lblHandName: TLabel;
    procedure FormCreate(Sender: TObject);
```

```
    procedure FormDestroy(Sender: TObject);
    procedure btnBetClick(Sender: TObject);
    procedure btnDrawClick(Sender: TObject);

  private
    { Private declarations }
    cardImages: array[1..5] of TImage;
    cardLabels: array[1..5] of TLabel;
    procedure CardClick(Sender: TObject);
  public
    { Public declarations }
  end;

var
  formMain: TformMain;
```

The primary items of interest in the interface section are the declarations of the two arrays of controls. The **cardImages** array will be used to display the playing card graphics. The **cardLabels** array will display information about the status of a card. It will be visible and display the message “Held” if the card is currently held.

```
implementation

{$R *.DFM}
{$R CACARDS03.RES}

uses
  caaPSCrd, caaPSDck, caaPS5CH;
```

Any time you use the Poker Suite, you will probably want **caaPSCrd**, **caaPSDck**, and at least one of the hand evaluation libraries. In this case we will be using **caaPS5CH**, the five-card library. These three units have been included in the **uses** clause.

```
const
  { minimum rating of a Jacks or Better poker hand }
  caa5CardJacksOrBetter  = $1BB432; { 1815602 }
```

Here we begin by defining a few constants. The five-card library does not define the value for a Jacks or Better poker hand, so we define it here. This will be used to determine if a hand qualifies for a (one for one) payback as a Jacks or Better hand. For the other hands, we will use the predefined constants in **caaPS5CH.pas**. (Incidentally, the value used in the constant

was obtained using the **HandRat5** program. I simply set up a hand with two Jacks, a Four, a Three, and a Two and had the program evaluate it.)

```
  { indexes into payout array }
  poNoPair        = 0;
  poJacks         = 1;
  poTwoPair       = 2;
  poThreeKind     = 3;
  poStraight      = 4;
  poFlush         = 5;
  poFullHouse     = 6;
  poFourKind      = 7;
  poStraightFlush = 8;
  poRoyalFlush    = 9;

  { lblInfo Messages - with leading space }
  PlaceBetCaption    = ' Place a Bet.';
  HoldDrawCaption    = ' Click Cards to Hold, then Draw.';

  { lblBet and lblCredit prefixes - with leading space }
  BetLabelCaption    = ' Bet: ';
  CreditLabelCaption = ' Credits: ';

  { lblHandName win and lose prefixes - no leading space }
  LoserMessage       = 'Too Bad! - ';
  WinnerMessage      = 'You Won! - ';
```

The program continues defining constants. The first group, are a set of array indexes into a new data type we will soon be defining. The string constants contain messages (or message prefixes) that will be displayed to the player.

```
type
  { array type to hold payouts }
  TPayOut = array[poNoPair..poRoyalFlush] of integer;

const
  { standard payouts five coin bet on 9-6 Jacks or Better.}
  StdPayOuts: TPayout = ( 0, 5, 10, 15, 20, 30, 45,
                          125, 250, 1250 );
```

Next, the new data type, **TPayOut** is defined and a typed constant is declared that holds all the game payout amounts. The game allows the player to bet in five credit increments, so the payouts reflect the paybacks for a five-credit bet. The game plays 9-6 Jacks or Better video poker, meaning that it pays back nine for one on a full house and six for one on a flush.

```
{ video poker game definition }
type
  TGameMode = (betMode, drawMode);
  TvidPokerGame = record
    Deck    : TcaaCardDeck;
    Hand    : Tcaa5CardHand;
    Bet     : integer;
    Credits : integer;
    PayOuts : TPayOut;
    Mode    : TGameMode;
  end;

var
  Game: TvidPokerGame;
```

The game mode type is now defined. The game will always be in one of two modes – bet mode or draw mode. In bet mode, the player is only allowed to click the "Bet" button or quit. In draw mode, the player may choose cards to hold, click on the "draw" button, or quit. The player may choose cards by clicking on the desired card image. When this is done in draw mode, the card label beneath the card will appear indicating the card is to be held. If the card is already marked as held, the card label will disappear. The game uses the "**Visible**" property of the card labels to determine if a card is held or not held.

The video poker game type (**TvidPokerGame**) is also defined here along with a global variable of this type. The variable **Game** will contain all pertinent game information while the program runs. It contains the card deck and the hand to be displayed and evaluated. It keeps track of the current **Bet**, **Credits**, and **Mode**. And it also stores the values of each payout.

```
{ function ImageName }
function ImageName(Card: TcaaCard): string;
begin
  case Card.CardSuit of
    caaClubs    : Result := 'CLUB_';
    caaDiamonds : Result := 'DIAMOND_';
    caaHearts   : Result := 'HEART_';
    caaSpades   : Result := 'SPADE_';
  end;
  if Card.CardRank < caaTen then Result := Result + '0';
  { aces default to high, so aces are a special case.
    if we don't compensate, we can return results
```

```
    like 'SPADE_14'. }
  if Card.CardRank = caaAceHigh then
    Result := Result + '01'
  else
    Result := Result + intToStr(ord( Card.CardRank) + 1 );
end;
```

The **ImageName** function returns the name of a card as it appears in the resource file. You will note that this version is slightly different than the version in the **TestCard** program. In this library, the deck is built with the aces set to **caaAceHigh**. The resource files are set up with the aces as **SUIT_01**. In the **TestCard** program, we compensated for that by setting them all to **caaAceLow**. This program doesn't do that. Instead, this routine tests for an ace set to the higher value and if one is found, it adds "01" to the suit prefix. (*Delphi 1 users should note that the suit prefixes are slightly different in that version.*)

```
{ procedure CardClick }
procedure TformMain.CardClick(Sender: TObject);
begin
  { in betting mode, clicking on the cards should have no
effect }
  if Game.Mode = betMode then exit;
  with Sender as TImage do
    CardLabels[Tag].Visible := not CardLabels[Tag].Visible;
end;
```

The **CardClick** procedure serves as the **Click** event handler for all the cards. It first checks the game mode. If the game mode is **betMode**, it exits. If the game mode is not **betMode**, the routine toggles the visibility setting of the card label associated with the card. The program uses this visibility setting to determine whether or not to hold a card. (*Delphi .NET users should note that the code is slightly different in the .NET versions. The **Tag** property under .NET is no longer an integer.*)

```
{ procedure FormCreate }
procedure TformMain.FormCreate(Sender: TObject);
var
  i,
  cardLeft,
  cardTop,
  cardWidth,
  cardHeight,
  cardSpace,
  labelTop,
  labelHeight: integer;
begin
  { game data - fill deck. set default starting values. }
  caaBuildCardDeck( Game.Deck );
  Game.Credits := 100;
  Game.Bet     := 0;
  Game.PayOuts := StdPayOuts;
  Game.Mode    := betMode;
```

The **FormCreate** procedure (***FormShow*** *in .NET versions*) is used to initialize everything used in the program and set the positions of the controls. It first declares a series of integer variables used to control the positioning loop and to set the control positions. This is a bit tedious but it permits us to space the controls evenly.

```
  { form information }
  formMain.Width := 604;
  formMain.Height := 284;

  { card position information }
  cardWidth  := 100;  { width of a card image }
  cardHeight := 140;  { height of a card image }
  cardSpace  := 16;   { space between card images }
  cardTop    := 8;    { top position of all card images }
  cardLeft   := 17;   { left position of first card image }
```

Here, the position variables are initialized and the form size is set.

```
  { information labels }
  lblBet.Left         := cardLeft;
  lblBet.Caption      := BetLabelCaption + '0';
  lblCredits.Left     := lblBet.Left + lblBet.Width + 4;
  lblCredits.Caption  := CreditLabelCaption + '100';
  lblInfo.Left        := cardLeft;
```

```
  lblInfo.Caption     := PlaceBetCaption;
  lblHandName.Left    := cardLeft;
  lblHandName.Caption := '';

  { card label position and size information }
  labelHeight := 24;
  labelTop    := cardTop + cardHeight;

  { game buttons }
  btnBet.Enabled := True;
  btnDraw.Enabled := False;
```

Next, the single instance controls, including the buttons and the three information labels are positioned and/or enabled.

```
  { place the images and labels }
  for i := 1 to 5 do
  begin
    cardImages[i]             := TImage.Create(formMain);
    cardImages[i].Parent      := formMain;
    cardImages[i].AutoSize    := False;
    cardImages[i].Stretch     := True;
    cardImages[i].Transparent := True;
    cardImages[i].Top         := cardTop;
    cardImages[i].Height      := cardHeight;
    cardImages[i].Width       := cardWidth;
    cardImages[i].Left        := cardLeft;
    cardImages[i].Tag         := i;
    cardImages[i].OnClick     := CardClick;
    cardImages[i].Picture.Bitmap.LoadFromResourceName(
      hInstance,'CARDBACK_01' );

    cardLabels[i] := TLabel.Create(formMain);
    cardLabels[i].Parent := formMain;
    cardLabels[i].AutoSize := False;
    cardLabels[i].Transparent := False;
    cardLabels[i].Color := clBlue;
    cardLabels[i].Font.Color := clAqua;
    cardLabels[i].Font.Name := 'Courier New';
    cardLabels[i].Font.Size := 14;
    cardLabels[i].Font.Style := [fsBold];
    cardLabels[i].Top := labelTop;
    cardLabels[i].Height := labelHeight;
    cardLabels[i].Width := cardWidth - 7;
    CardLabels[i].Left := cardLeft + 4;
    cardLabels[i].Caption := 'HELD';
```

```
    cardLabels[i].Alignment := taCenter;
    cardLabels[i].Visible := False;
    cardLabels[i].Tag := i;

    { set position for next card }
    cardLeft := cardLeft + cardSpace + cardWidth;
  end;
end;
```

Finally, the initialization and positioning loop for the card images and card labels places those components on the form and assigns their settings. The game will initially display the cards with red card back images.

```
{ procedure FormDestroy
  Does: Frees the cardImages and cardLabels. }
procedure TformMain.FormDestroy(Sender: TObject);
var
  i: integer;
begin
  { free the controls we created }
  for i := 1 to 5 do
  begin
    CardImages[i].Free;
    CardLabels[i].Free;
  end;
end;
```

Since the card images and labels were created in code, they are freed in the **FormDestroy** procedure.

```
{ procedure btnBetClick }
procedure TformMain.btnBetClick(Sender: TObject);
var
  cd : integer;
  CardImageName: string;
  pCardName: array[0..10] of char;
begin
  lblHandName.Caption := '';
  Game.Credits       := Game.Credits - 5;
  lblCredits.Caption :=
    CreditLabelCaption + IntToStr( Game.Credits );
  Game.Bet       := 5;
  lblBet.Caption := BetLabelCaption + '5';
  caaShuffleCardDeck( Game.Deck );
```

The **Click** events for the “Bet” and “Draw” buttons handle most of the work in the program. The **btnBetClick** procedure handles betting and dealing the cards to the player. Its first jobs are fairly simple. It clears the label containing the hand name and win/loss message. It next subtracts five from the current credits and changes the credit label display. It then sets the bet to five and changes the bet label display. The deck is shuffled and we are ready to deal the cards.

```
  for cd := 1 to 5 do
  begin
    Game.Hand.Cards[cd] := Game.Deck[cd-1];
    { display  card images }
    CardImageName := ImageName( Game.Hand.Cards[cd] );
    StrPCopy(pCardName,CardImageName);
    cardImages[cd].Picture.Bitmap.LoadFromResourceName(
     hInstance,pCardName );
  end;
```

The first five cards in the card deck are copied into the hand. The hand is then displayed. To do this, the routine gets the image name of the card in the resource file and uses that to load the bitmap into the card image component. (*The .NET code is somewhat different.*)

```
  lblInfo.Caption := HoldDrawCaption;
  btnBet.Enabled := False;
  btnDraw.Enabled := True;

  Game.Mode := drawMode;
end;
```

The information label (**lblInfo**) is used to let the player know what action should be performed. Since we are about to change to draw mode, the label is updated to reflect this. Next the “Bet” button is temporarily disabled and the “Draw” button is enabled. Finally the game mode is changed to draw mode.

```
{ procedure btnDrawClick }
procedure TformMain.btnDrawClick(Sender: TObject);
var
  cd : integer;
  CardImageName: string;
  pCardName: array[0..10] of char;
  winStr: string;
```

Drawing cards is a bit more complex than betting. The **btnDrawClick** routine is longer and more complex than the Click event for the "Bet" button. We first declare a set of local variables. These are the same as in **btnBetClick** with one addition. The **winStr** variable will be used to display the winning or losing message to the player.

```
begin
  for cd := 1 to 5 do
  begin
    { if cardLabel is not visible then card is not held. }
    if not cardLabels[cd].Visible then
    begin
      { get a replacement card }
      Game.Hand.Cards[cd] := Game.Deck[cd+4];
      { display  card images }
      CardImageName := ImageName( Game.Hand.Cards[cd] );
      StrPCopy(pCardName,CardImageName);
      cardImages[cd].Picture.Bitmap.LoadFromResourceName(
        hInstance,pCardName );
    end
    else { label is visible - change this }
      cardLabels[cd].Visible := False;
  end;
```

What we need to do is discard any cards that player did not hold. We loop through the five cards and check to see if the corresponding card label is currently visible. If it is, the player wants to retain this card and all we need to do is make the label invisible.

If the label was ***not visible***, the player the player wants the card replaced. Video poker programs typically do this in one of two ways. The next available card in the deck can be "dealt" into the first open position. The card after that can then be dealt into the second position and so on until all the discarded cards are replaced. This method is slightly tricky to implement.

An easier and safer method is to simply replace the card in the hand with one at a fixed offset in the deck. This is what the code above does. This has the advantages that it is simpler to implement and less likely to produce bugs. ***Note, that this does not change the player's odds in any way.***

```
  { get hand rating }
  caa5CardSetHighValue( Game.Hand );

  if Game.Hand.Rating < caa5CardJacksOrBetter then
  begin
    winStr := LoserMessage;
  end
  else if Game.Hand.Rating < caa5CardTwoPair then
  begin
    { Jacks or Better }
    Game.Credits := Game.Credits + Game.Payouts[poJacks];
    winStr := WinnerMessage;
  end
  else if Game.Hand.Rating < caa5CardThreeOfAKind then
  begin
    { two pair }
    Game.Credits := Game.Credits + Game.Payouts[poTwoPair];
    winStr := WinnerMessage;
  end
  else if Game.Hand.Rating < caa5CardStraight then
  begin
    { three of a kind }
    Game.Credits := Game.Credits +
                    Game.Payouts[poThreeKind];
    winStr := WinnerMessage;
  end
  else if Game.Hand.Rating < caa5CardFlush then
  begin
    { straight }
    Game.Credits := Game.Credits +
                    Game.Payouts[poStraight];
    winStr := WinnerMessage;
  end
  else if Game.Hand.Rating < caa5CardFullHouse then
  begin
    { flush }
    Game.Credits := Game.Credits + Game.Payouts[poFlush];
    winStr := WinnerMessage;
  end
  else if Game.Hand.Rating < caa5CardFourOfAKind then
  begin
    { full house }
    Game.Credits := Game.Credits +
Game.Payouts[poFullHouse];
    winStr := WinnerMessage;
  end
```

```
  else if Game.Hand.Rating < caa5CardStraightFlush then
  begin
    { four of a kind }
    Game.Credits := Game.Credits +
                    Game.Payouts[poFourKind];
    winStr := WinnerMessage;
  end
  else if Game.Hand.Rating < caa5CardRoyalFlush then
  begin
    { straight flush }
    Game.Credits := Game.Credits +
                    Game.Payouts[poStraightFlush];
    winStr := WinnerMessage;
  end
  else
  begin
    { royal flush }
    Game.Credits := Game.Credits +
                    Game.Payouts[poRoyalFlush];
    winStr := WinnerMessage;
  end;
```

The lengthy bit of code above evaluates the hand and then uses the high rating obtained to determine whether the player has won or lost. If the rating is less than the rating constant we assigned to a Jacks or Better hand, the player has lost and we only need to assign the losing prefix to the **winStr** variable. If the player has one of the winning hands, the code adds the appropriate payout amount from **Game.Payouts** to the current credits and assigns the winning prefix to **winStr**.

The order that the decisions were made in is from most likely to least likely. This is theoretically more efficient and helps ensure that nothing is left out. But it has the undesirable effect of making the code slightly obtuse. ***We check to see if a rating is less than that of the next higher hand.*** Thus (for example) to check whether a hand is a full house, we actually check to see whether the rating is less than that for a four of a kind hand. It is therefore important that the choices be made in the proper order.

```
  { update the display labels }
  lblBet.Caption      := BetLabelCaption + '0';
  lblInfo.Caption     := PlaceBetCaption;
  lblCredits.Caption  :=
    CreditLabelCaption +
    IntToStr( Game.Credits );
```

```
  lblHandName.Caption :=
    { IntToHex( Game.Hand.Rating, 6 ) + ' ' + }
    winStr +
    caa5CardGetHandName( Game.Hand );

  { update buttons }
  btnBet.Enabled  := True;
  btnDraw.Enabled := False;

  Game.Mode := betMode;
end;

end.
```

The rest of the code updates the various labels, the enabled status of the buttons, and the current game mode. The bet label is set to the bet label prefix plus “0”. The general information label is updated to reflect that we are back in **betMode**. The credits label is changed to reflect the current amount of credits.

The program next assigns the **winStr** variable to the hand name label and adds the name of the hand to it as well. If you removed the comments from around the **IntToHex** function, the label would also display the hand rating in hexadecimal format. This is useful during debugging (should you decide to modify the program).

Finally, since we are going back into betting mode, the “Bet” label is re-enabled and the “Draw” button is disabled. Last, but not least, the modes is changed to **betMode**.

Appendix A – Alphabetical List of Library Routines.

```
caa3CardGetHandName( const Hand:Tcaa3CardHand ):
                     Tcaa3CardHandName;
```

Type of Routine: Function
Declared in: caaPS3CH.pas (see also page 21)
Expects: A variable or constant of type **Tcaa3CardHand** with the **Rating** field set.
Returns: The name of a hand.
Notes: The **Rating** field of the Hand ***must be set*** prior to calling this function. If this is not done, the hand name will contain an error message instead of the name of the hand.

```
caa3CardSetHighValue( var Hand: Tcaa3CardHand );
```

Type of Routine: Procedure
Declared in: caaPS3CH.pas (see also page 21)
Expects: A variable or type **Tcaa3CardHand.**
Does: Sets the Rating field to the high hand rating.
Notes: The rating of a hand should always be set before calling a hand naming function.

```
caa3CardSetLowValue( var Hand: Tcaa3CardHand );
```

Type of Routine: Procedure
Declared in: caaPS3CH.pas (see also page 21)
Expects: A variable or type **Tcaa3CardHand.**
Does: Sets the Rating field to the low hand rating.
Notes: The rating of a hand should always be set before calling a hand naming function.

```
caa4CardGetHandName( const Hand:Tcaa4CardHand ):
                     Tcaa4CardHandName;
```

Type of Routine: Function
Declared in: caaPS4CH.pas
Expects: A variable or constant of type **Tcaa4CardHand** with the **Rating** field set.
Returns: The name of a hand.
Notes: The **Rating** field of the Hand ***must be set*** prior to calling this function. If this is not done, the hand name will contain an error message instead of the name of the hand.

```
caa4CardSetHighValue( var Hand: Tcaa4CardHand );
```

Type of Routine: Procedure
Declared in: caaPS4CH.pas
Expects: A variable or type **Tcaa4CardHand.**
Does: Sets the Rating field to the high hand rating.
Notes: The rating of a hand should always be set before calling a hand naming function.

```
caa4CardSetLowValue( var Hand: Tcaa4CardHand );
```

Type of Routine: Procedure
Declared in: caaPS4CH.pas
Expects: A variable or type **Tcaa4CardHand.**
Does: Sets the Rating field to the low hand rating.
Notes: The rating of a hand should always be set before calling a hand naming function.

```
caa5CardGetHandName( const Hand:Tcaa5CardHand ):
                        Tcaa5CardHandName;
```

Type of Routine: Function
Declared in: caaPS5CH.pas
Expects: A variable or constant of type **Tcaa5CardHand** with the **Rating** field set.
Returns: The name of a hand.
Notes: The **Rating** field of the Hand ***must be set*** prior to calling this function. If this is not done, the hand name will contain an error message instead of the name of the hand.

```
caa5CardSetHighValue( var Hand: Tcaa5CardHand );
```

Type of Routine: Procedure
Declared in: caaPS5CH.pas
Expects: A variable or type **Tcaa5CardHand.**
Does: Sets the Rating field to the high hand rating.
Notes: The rating of a hand should always be set before calling a hand naming function.

```
caa5CardSetLowValue( var Hand: Tcaa5CardHand );
```

Type of Routine: Procedure
Declared in: caaPS5CH.pas
Expects: A variable or type **Tcaa5CardHand.**
Does: Sets the Rating field to the low hand rating.
Notes: The rating of a hand should always be set before calling a hand naming function.

```
caaBuildCardDeck( var Deck: TcaaCardDeck );
```

Type of Routine: Procedure
Declared in: caaPSDck.pas (see also page 13)
Expects: A variable or type **TcaaCardDeck.**
Does: Fills the Deck variable with a complete set of playing cards, ordered from the Ace of Clubs to the King of Spades.
Notes: This procedure should always be called prior to using a deck.

```
caaGetCardName( Card: TcaaCard ): TcaaCardName;
```

Type of Routine: Function
Declared in: caaPSCrd.pas (see also page 6)
Expects: A variable or constant of type **TcaaCard**.
Returns: The name of a playing card.

```
caaSetCard( var Card: TcaaCard;
                Rank: TcaaCardRank;
                Suit: TcaaCardSuit );
```

Type of Routine: Procedure
Declared in: caaPSCrd.pas (see also page 6)
Expects: A variable or type **TcaaCard**, a variable or constant of type **TcaaCardRank**, and a variable or constant of type **TcaaCardRank**.
Does: Sets the **CardRank** and **CardSuit** fields of a playing card.

```
caaSetCardRank( var Card: TcaaCard;
                    Rank: TcaaCardRank );
```

Type of Routine: Procedure
Declared in: caaPSCrd.pas (see also page 6)
Expects: A variable or type **TcaaCard** and a variable or constant of type **TcaaCardRank**.
Does: Sets the **CardRank** field of a playing card.

```
caaSetCardSuit( var Card: TcaaCard;
                    Suit: TcaaCardSuit );
```

Type of Routine: Procedure
Declared in: caaPSCrd.pas (see also page 6)
Expects: A variable or type **TcaaCard** and a variable or constant of type **TcaaCardSuit**.
Does: Sets the **CardSuit** field of a playing card.

```
caaShuffleCardDeck( var Deck: TcaaCardDeck );
```

Type of Routine: Procedure
Declared in: caaPSDck.pas (see also page 13)
Expects: A variable or type **TcaaCardDeck.**
Does: Shuffles the Deck. The order of the cards will now be (more or less) random.
Notes: Make sure to set up the deck by calling **caaBuildCardDeck** before shuffling the first time.

(The rest of this page was intentionally left blank.)

Appendix B – Alphabetical List of Custom Data Types

```
Tcaa3CardArray = array[Tcaa3CardHandPosition] of TcaaCard;
```
Purpose: An array type used to define the three-card poker hand. The three-card poker hand is a one-based array of three elements of **TcaaCard** type.
Declared in: caaPS3CH.pas (see also page 21)

```
Tcaa3CardHandPosition = 1..3;
```
Purpose: A sub-range type used to define the lower and upper bounds of the three-card poker hand array.
Declared in: caaPS3CH.pas (see also page 21)

```
Tcaa3CardHandRating = LongInt;
```
Purpose: An alias for the **LongInt** data type. The hand rating is a 32-bit integer that contains a numeric evaluation of the hand. It consists of a base rank value followed by the ranks of the individual cards. It is used to determine which hand has the best value (low or high).
Declared in: caaPS3CH.pas (see also page 21)

```
Tcaa3CardHandName = TcaaCardName;
```
Purpose: An alias for the **TcaaCardName** data type.
Declared in: caaPS3CH.pas (see also page 21)

```
Tcaa4CardArray = array[Tcaa4CardHandPosition] of TcaaCard;
```
Purpose: An array type used to define the four-card poker hand. The four-card poker hand is a one-based array of four elements of **TcaaCard** type.
Declared in: caaPS4CH.pas

```
Tcaa4CardHandPosition = 1..4;
```
Purpose: A sub-range type used to define the lower and upper bounds of the four-card poker hand array.
Declared in: caaPS4CH.pas

```
Tcaa4CardHandRating = LongInt;
```
Purpose: An alias for the **LongInt** data type. The hand rating is a 32-bit integer that contains a numeric evaluation of the hand. It consists of a base rank value followed by the ranks of the individual cards. It is used to determine which hand has the best value (low or high).

Declared in: caaPS4CH.pas

```
Tcaa4CardHandName = TcaaCardName;
```
Purpose: An alias for the **TcaaCardName** data type.
Declared in: caaPS4CH.pas

```
Tcaa5CardArray = array[Tcaa5CardHandPosition] of TcaaCard;
```
Purpose: An array type used to define the five-card poker hand. The five-card poker hand is a one-based array of five elements of **TcaaCard** type.
Declared in: caaPS4CH.pas

```
Tcaa5CardHandPosition = 1..5;
```
Purpose: A sub-range type used to define the lower and upper bounds of the five-card poker hand array.
Declared in: caaPS5CH.pas

```
Tcaa5CardHandRating = LongInt;
```
Purpose: An alias for the **LongInt** data type. The hand rating is a 32-bit integer that contains a numeric evaluation of the hand. It consists of a base rank value followed by the ranks of the individual cards. It is used to determine which hand has the best value (low or high).
Declared in: caaPS5CH.pas

```
Tcaa5CardHandName = TcaaCardName;
```
Purpose: An alias for the TcaaCardName data type.
Declared in: caaPS5CH.pas

```
TcaaCard = record
  CardRank : TcaaCardRank;
  CardSuit : TcaaCardSuit;
end;
```
Purpose: A record type defining playing cards.
Declared in: caaPSCrd.pas (see also page 6)

```
TcaaCardName = string[32];    { 16-bit and 32-bit }
TcaaCardName = string;        { .NET }
```
Purpose: Alias for **string** or **string**[32] depending on the version.
Declared in: caaPSCrd.pas (see also page 6)

```
TcaaCardRank =
  ( caaAceLow, caaTwo, caaThree, caaFour,
    caaFive, caaSix, caaSeven, caaEight,
    caaNine, caaTen, caaJack,  caaQueen,
    caaKing, caaAceHigh );
```

Purpose: An enumerated type used to define the rank of playing cards.
Declared in: caaPSCrd.pas (see also page 6)

```
TcaaCardSuit =
  ( caaClubs, caaDiamonds, caaHearts,
    caaSpades );
```

Purpose: An enumerated type used to define the suit of playing cards.
Declared in: caaPSCrd.pas (see also page 6)

```
TcaaDeckRange = caaClubAce..caaSpadeKing;
```

Purpose: A sub-range type used to define the lower and upper bounds of the poker deck array.
Declared in: caaPSDck.pas (see also page 13)

```
TcaaCardDeck = array[TcaaDeckRange] of TcaaCard;
```

Purpose: An array type used to define the poker deck. The Card Deck is a zero-based array of fifty-two elements of **TcaaCard** type.
Declared in: caaPSDck.pas (see also page 13)

Appendix C – Differences and Installation Notes

There are four different Poker Suite packages available, plus document and card resource files. The one(s) you wish to download and install depend on the version(s) Delphi you are using. This section explains the differences between the versions and how to install them.

All of the 32-bit versions of the Poker Suite are the same. However, the demo and tutorial packages differ slightly. This is because Delphi 2 and Delphi 3 do not have all the features of the later versions.

Documents: All Users – (spsdoc.zip)

All users should download the library documentation file (spsdoc.zip). This contains the library documentation in Word and PDF formats.

Delphi 1 – (spsd1.zip and cards16.zip)

The Delphi 1 version of the library is contained in the file “spsd1.zip”. To install, simply unzip the file into a work directory.

The code for the 16-bit library differs from the other versions. The differences lie in the naming conventions used in the resource files, the names of those files, and in the hand naming function in the five-card hand unit. The first two differences are explained in the text.

The third difference is the result of a minor incompatibility between the 16-bit version of Delphi and the other versions. In Delphi 1 a case selector cannot be a 32-bit value. Therefore, the hand naming routine uses if/then/else statements instead.

The three 16-bit card resource files are contained in the file “Cards16.zip”. Unzip these into the same directory as the library.

Delphi 2 and 3 – (spsd23.zip and cards32.zip)

The Delphi 2 and Delphi 3 versions of the library are contained in the file “spsd23.zip”. To install, simply unzip the file into a work directory.

The code for the library files is the same as the other 32-bit versions even though it is marked as being for Delphi 2 and 3. The demo programs are

slightly different than the other 32-bit demos – due mainly to property differences in the components used.

The three 32-bit card resource files are contained in the file “Cards32.zip”. Unzip these into the same directory as the library.

Delphi 4 through 2005 – (spsd49.zip and cards32.zip)

The Delphi 4 through Delphi 7, and Delphi 2005 (Naïve 32-Bit) versions of the library are contained in the file “spsd49.zip”. To install, simply unzip the file into a work directory.

Delphi 2005 Note: No “.bdsproj” files are included with this package. To load a project in Delphi 2005, load the “.dpr” file. Delphi will tell you the project must be upgrades, and ask if you wish to upgrade it to Windows 32-bit or .NET. Choose Windows 32-Bit. Delphi 2005 will create a “.bdsproj” file.

The three 32-bit card resource files are contained in the file “Cards32.zip”. Unzip these into the same directory as the library.

Delphi 8 and 2005 (.NET) – (spsdnet.zip and cards32.zip)

The “Delphi for .NET” versions of the library are contained in the file “spsdnet.zip”. To install, unzip the file into a work directory.

The library defines two sting types. In the 16-bit and 32-bit versions these were defined as **`string[32]`**. In the .NET versions they are defined as **`string`**. The demo programs in this version are VCL.NET applications.

The three 32-bit card resource files are contained in the file “Cards32.zip”. Unzip these into the same directory as the library.

Delphi 2005 Users: The demo and utility projects were originally created using Delphi 8. The “**`.dpr`**” files contain path information that may (or may not) cause problems. For example, the program file “ht3Card.dpr” contains the following:

```
{%DelphiDotNetAssemblyCompiler 'c:\program files\common files\Borland
 shared\bds\shared assemblies\2.0\Borland.Vcl.dll'}
{%DelphiDotNetAssemblyCompiler 'c:\program files\common files\borland
 shared\bds\shared assemblies\2.0\Borland.Delphi.dll'}
{%DelphiDotNetAssemblyCompiler 'c:\program files\common files\borland
```

```
 shared\bds\shared assemblies\2.0\Borland.VclRtl.dll'}
```

Note that each of the three paths is actually a single line in the file. You can see that these paths reference "**…\shared assemblies\2.0\…**". This has the potential to cause a problem. If you don't have Delphi 8 installed on your computer, this path may not exist and I don't know if D2005 changes this information on machines on which Delphi 8 has not been installed. The simple solution is to change the "**\2.0**" to "**\3.0**".